An exploration of machine learning methods on Space Titanic

Thale Bjerkreim[1], Eirik Pedersen[1], and Aurora Bjørnerud[1]

{thbje9813, eiped4842, aubjo4239}@uib.no

Abstract

In this paper we present our submission to the Spaceship Titanic Kaggle-competition. The task is a binary classification problem where we want to predict if a passenger was transported to another dimension after the spaceship collided with a spacetime anomaly. We present novel techniques for imputing missing data, improving diversity within tree-ensemble methods and testing a pretrained transformer for classification. The ensembles and transformer are compared to other standard machine learning models with accuracy as the performance measure. The results show a small yet general better performance on the models trained on the preprocessed data rather than the augmented data.[1] The overall best model was the pretrained transformer fitted on the preprocessed data.

1 Introduction

Problem description

The intention of this paper is to explore the performance of different machine learning methods on the Spaceship Titanic competition on Kaggle [1]. This is a binary classification problem where our task is to predict if a passenger has been transported to another dimension as a result of the spaceship's collision with the anomaly. We are given a training dataset and a test dataset, each with 13 features with six of them being categorical and seven numerical, and lastly a sparse distribution of missing values. A significant part of the task is therefore how to handle the missing data in terms of optimizing performance.

Related works

The Spaceship Titanic competition has been ongoing since 2022 with 2424 participants to date [1]. It is considered a standard machine learning problem that Kaggle recommends to beginners in order to become familiar with machine learning and Kaggle competitions. Ignoring the currently best entry of 96% accuracy score, the best performances are at 84% or lower. Exploring different submissions we find that others have tested models such as Gradient Boosting Classifiers, Random Forest, KNN and SVC among many others. The simplicity of the tasks makes it the perfect candidate for a vast amount of models as shown in the submissions [1].

Objectives

Since the problem itself is a standard machine learning problem we put our focus how to preprocess the data in the most optimal way to achieve better results. We investigate whether standard machine learning methods and preprocessing methods can outperform newer ideas on the subjects.

Contributions

We explore three different ideas:
1. Augmenting the data by imputing values using an automated diffusion-based pipeline to deal with missing entries, and using the same models to generate synthetic data in order to improve generalization.
2. Applying random rotations to the base learners within different tree based ensemble methods to increase diversity between them
3. Implementing TabPFN, because it has been shown to achieve strong results on smaller scale classification problems.

2 Methods

Exploratory data analysis and preprocessing

We started the project by looking at our data. The training dataset has 8693 datapoints. It has an even class distribution with 4378 labels being true and 4315 being false. With 24% of the rows containing NaN values, the preprocessing is essential for achieving good results. We did a combination of imputing, removing and manual feature engineering based on patterns observed in the data.

Normally, data is split into training and test sets, and furthermore the test set is split into validation and test sets. Since the Kaggle test set does not contain the label, we split the provided training data 80/20 into training and validation sets. We tuned the hyperparameters using the validation set

[1] Both the "preprocessed" and "agumented" data are training data that have been preprocessed. Their names come from the "augmented" one including some particular imputation steps that generate synthetic data.

Proceedings of the 8th Northern Lights Deep Learning Conference (NLDL), PMLR V, 2027.

and retrained the final model on the whole training set(validation + training). This is a common practice in Kaggle competitions because it enables optimal hyperparameter selection while maximizing the amount of data used for training the model.

Evaluation and selection

The metric used in the competition is accuracy score. This metric could give misleading results for imbalances datasets, but this is not an issue in this competition because, as mentioned previously, the label distribution is even. For each model other than TabPFN we trained two versions: one version was trained on the preprocessed dataset, and the other was trained on the dataset using Forest Diffusion[2] imputation and generation. Three seperate TabPFN models were trained, one on the raw data provided by Kaggle, one on the preprocessed data and one on the augmented data. We used GridSearchCV to find the best hyperparameters with a predefined split and set refit=True. After the validation set was used to find the best hyperparameters we could then train the models on both the training and the validation set. This was the workflow for all models except for TabPFN. Since we used predefined split the GridSearchCV score was simply the accuracy score on our validation set. We stored the validations scores for each model, chose the model that got the highest validation score and evaluated it on the unseen test dataset.

2.1 Forest Diffusion Synthetic Data Generation and Imputation

[2] Overfitting is a common challenge for small datasets which can be dealt with in different ways, including generating synthetic data. For tabular data, tree-based methods are known to perform well, therefore an XGBoost-based data imputation and augmentation pipeline using Forest Diffusion is a sound candidate. This method has been shown to be competitive with leading modern methods, [2] and is specifically targeted at mixed-type tabular data, which makes it suitable in this use case.

2.1.1 Implementation

For this project, the algorithms are manually implemented as described in the paper "Generating and Imputing Tabular Data via Diffusion and Flow-based Gradient-Boosted Trees,"[2] still the authors have also released an open source Gihtub repository containing their implementation. [3] This involves duplicating the dataset for diverse data/noise pairs, training explicit XGBoost models for each noise level to predict trajectories for denoising data, functions to add/remove noise, and the imputation and generation process in itself.

The XGBoost models are trained after the train/test split, meaning they are fit on the first 6954 observations. For the repaint jumps in the forest diffusion repaint imputation function, a jump interval of five noise levels and ten repaints is used which is ambiguous in the paper. Otherwise, variables are set to their standard values. Training the models requires randomness, so the models are provided with the load_models() function both for reproducibility and because of training time.

2.2 TabPFN

[3] There are many standard, easy to interpret methods that achieve good results on basic binary classification problems on small tabular datasets. We wanted to explore a more complex modern method and evaluate how it compared to the reliable methods that are typically chosen.

TabPFN, short for tabular prior fitted network, is a pretrained transformer that solves classification problems on small tabular datasets[4]. It is trained on synthetic data made by priors, which are data generating mechanisms in this context. Since we are solving a binary classification problem and the available datasets are relatively small, TabPFN is well suited to solve our problem. The first version of the model would most likely not yield good results because it was optimized for even smaller numerical datasets without NaN values. We used TabPFN version 2.5 which has improved on the earlier limitations. In version 2.5 the scalability is improved. They achieved SOTO performance and faster inference [5].

2.2.1 Implementation

Prior Labs have made TabPFN available to the community. We used the library TabPFN_client, which is an API that gives access to their cloud based models, where the input data is processed and predictions are generated[6]. Authentication was preformed by logging in to their website, where we got an access token which was given to their set_access_token method. The library itself is easy to use and makes the models accessible to non experts and experts alike. We simply defined the model, fitted it on the training dataset, made predictions on the validation set that contains the features and found the accuracy score based on the predicted labels for the validation set and the actual labels. This workflow is similar to more traditional machine learning libraries such as scikit-learn. We evaluated the model on preprocessed data, augmented data and raw data, and

[2]The sections on Forest Diffusion are the individual work of Eirik

[3]The sections on TabPFN are the individual work of Thale

compared the resulting accuracy scores. We wanted to asses the impact of preprocessing, since version 2.5 is suppose to handle categorical features and NaN values well.

2.3 Random Rotation Ensembles

[4] Given our problem being a standard classification problem, we decided to attempt solving it with some of the typical machine learning models, including decision trees and decision tree ensemble methods. In an attempt to further improve the performances of these models, we decided to implement the idea of randomly rotating the feature space of each base learner in an ensemble, proposed by Blaser et al. (2016)[7]. The idea behind these rotations comes from the problem of diversity vs. accuracy in ensemble methods. They key idea behind why ensemble methods often perform better than single models lies in the diversity between their base learners. If their base learners were to all be identical, the result would be no better than using a single model. However, measures made to ensure that the diversity is great enough often have a tendency to affect their individual performances negatively. The motivation for these random rotations is therefore to increase the diversity between base learners with minimal negative impact to their performances. Some important notes mentioned from the paper [7] are that these rotations are only to be applied on numerical data, and the rotations applied have to come from a uniform distribution as to not favor any rotation above another.

Implementation

We decided to test the rotations on Extra trees, Random Forests and an ensemble of Decision trees. To implement this we created a wrapper using ExtraTreeClassifier and DecisionTreeClassifier from the sklearn library. As the random rotation of the feature space is performed for each base learner, different means of application had to be done depending on how the ensemble methods sample from the training data. While each base learner in extra trees receive the entire training data set, the base learners in a Random forest only receive a bootstrap sample of the data. This makes implementing the rotation for extra trees somewhat simpler than for a Random forest, as we can simply rotate the training set once for each base learner and use an Extra Tree with one learner for each base learner. The bootstrapping in Random Forest is built into the model, meaning we won't know what data each base learner receives within the method. As a workaround for this issue, we simply created the bootstrap samples ourselves, randomly rotated them, and then passed them to Decision Trees as the base learners. By doing this we were able to simulate how the rotations would work on a Random Forest, without using the Random Forest class from sklearn. The random rotations were only applied to numerical features, found by taking all features with at least 10 distinct values. Furthermore, the rotation matrices for each base learner was stored in order to correctly rotate any new data for prediction.

[4]The sections on Random Rotation Ensembles are the individual work of Aurora

3 Results

3.1 Datasets

Fitting the best performing model for each dataset, the augmented dataset trained on an XGBoost model got an accuracy score of 0.80, AUC of 0.89, and F1 score of 0.79. The manually preprocessed, real only dataset trained on a TabPFN model got an accuracy score of 0.81, AUC of 0.90 and F1 score of 0.82. Graphical representations are shown in Fig. 1 and Fig. 2.

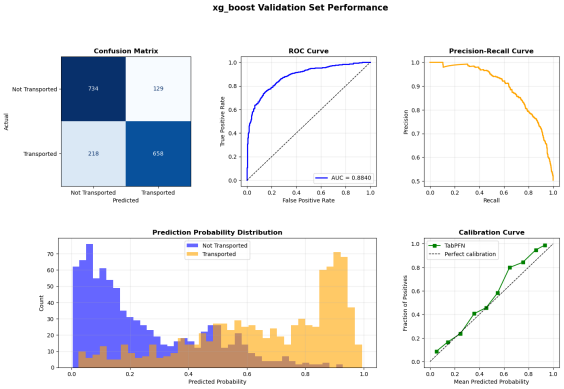

Figure 1. Augmented Dataset Metrics

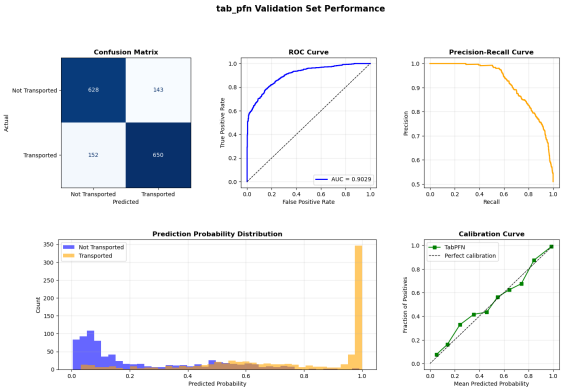

Figure 2. Manually Preprocessed Dataset Metrics

3.2 Model performances

In Fig. 3 we can see how all of our different models perform. The rotated and non-rotated versions of Random Forest and Extra trees appear to have a slight difference in performance on both the augmented and preprocessed data. This slight difference is a positive for random forest and a negative on for extra trees. This does not apply to the ensemble of decision trees where the randomly rotated one outperforms the non-rotated one with 2-3% difference for both datasets.

We can also see that the TabPFN model fitted on the preprocessed data achieves better results than the two other TabPFN models. This means that even though TabPFN 2.5 is capable of handling missing values and different feature types, the preprocessed data gave an additional advantage over raw data in this project. Comparing the three TabPFN models, the one that got the worst results was the model fitted on unchanged data from Kaggle.

There is a general pattern in the validation score results. Models trained on the normally preprocessed data gave generally better scores. The only models trained on augmented data that achieved better results were the baseline model, which is a dummyclassifier, and the randomly rotated decision trees. The dummyclassfier ignores the input, therefore in reality only one model was better when trained on the augmented data.

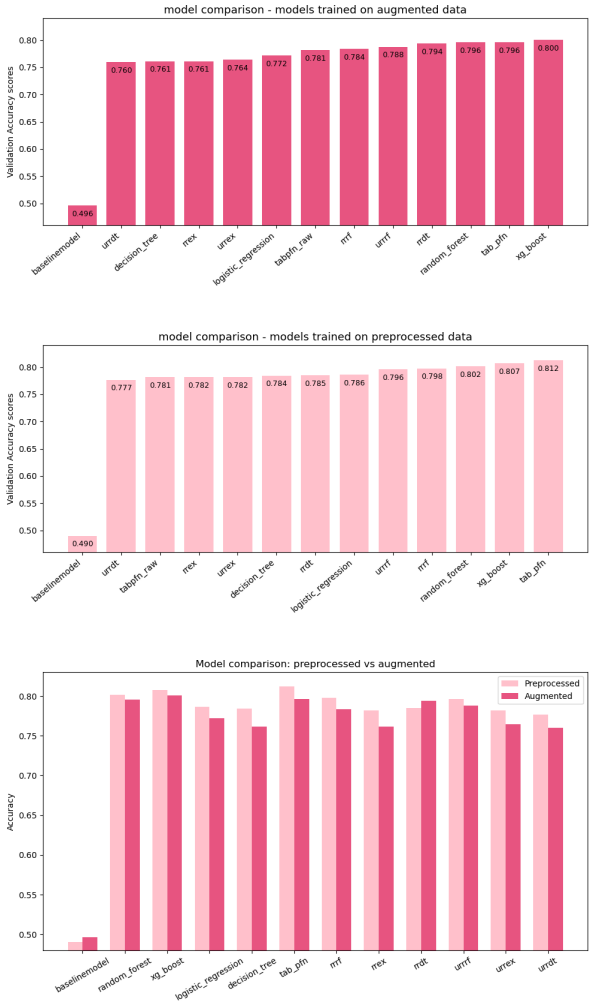

Figure 3. These figures show the validation scores of trained on augmented data and preprocessed data. There are three models that are both randomly rotated and not rotated. These start with "rr" for randomly rotated" and "urr" for the un-randomly rotated ones. The ones ending with "ex" are extra trees, the "rf" are random forests and "dt" are ensembles of decision trees.

4 Discussion

4.1 Dataset

Looking at Fig. 1 and Fig. 2 we can see that the ROC curve, precision-recall and calibration for both models look very similar, while the baseline has a slight edge on false negatives which results in the higher overall accuracy. The most interesting metric is the prediction probability distribution. The baseline model seems to be very confident in its yes-predictions while the model trained on augmented data has a more natural graph. It is generally able to separate transported/nots, yet places more samples in the 0.4-0.7 range of not-sure. This could mean that the augmented model generalizes better. However, it might also be that the model trained on observed data only has learned some relationship that gets lost in noise for the augmented dataset. Everything considered, it seems like the algorithm is working as intended, it simply didn't provide better results in this case.

4.2 Random Rotations

The difference in using the random rotations on random forests and extra trees versus the standard decision trees was not expected, yet seems reasonable. We suspect it being a result of how the first two models are inherently diverse. When adding the rotations to a random forest or a random tree it is simply one additional diversifying step to their already existing ones. While for the ensemble of decision trees, the rotation step is generally the only measure used to diversify them, therefore creating a larger effect.

Given the almost 50/50 distribution of numerical and categorical features, the generally minimal effect is not surprising. A better performance could perhaps be expected if distribution was heaver on

the numerical side, yet referring to the experiments of [7], it is not a guarantee.

A conclusion on our experiments with random rotation is that they appear to generally be more efficient when applied to a model with lower diversity. The effect becomes notably smaller when applied to models that already have methods for increasing it.

4.3 TabPFN

All the TabPFN models gave relatively good results, which is not a surprise. TabPFN has been shown to outperform treebased models and get accuracy scores that matches AutoGluon 1.4 [5]. XGboost does achieve better results than TabPFN on the augmented dataset. This aligns with the authors own findings that xgboost wins 13% of the time when trained on medium datasets(over 10 000 samples). The standard models in our project are trained on maximum 15,000 samples, whereas TabPFN is trained on a very large amount of synthetic data. Therefore the comparisons between these models might be unfair and does not yield very meaningful insights since the models are fundamentally different.

References

[1] A. C. Addison Howard and R. Holbrook. *Spaceship Titanic*. 2022. URL: https://www.kaggle.com/competitions/spaceship-titanic.

[2] A. Jolicoeur-Martineau, K. Fatras, and T. Kachman. "Generating and Imputing Tabular Data via Diffusion and Flow-based Gradient-Boosted Trees". In: (2024). Accessed: 2026-04-20. arXiv: 2309.09968.

[3] A. Jolicoeur-Martineau, K. Fatras, and T. Kachman. *Forest Diffusion Github Repository*. 2024. URL: https://github.com/SamsungSAILMontreal/ForestDiffusion (visited on 04/20/2026).

[4] K. E. Noah Hollmann Samuel Müller and F. Hutter. *TABPFN: A TRANSFORMER THAT SOLVES SMALL TABULAR CLASSIFICATION PROBLEMS IN A SECOND*. 2023. URL: https://openreview.net/pdf?id=cp5PvcI6w8_ (visited on 04/20/2026).

[5] P. L. Team. *TabPFN-2.5: Advancing the State of the Art in Tabular Foundation Models*. 2026. URL: https://arxiv.org/pdf/2511.08667 (visited on 04/20/2026).

[6] P. Labs. *tabpfn-client*. 2025. URL: https://github.com/PriorLabs/tabpfn-client (visited on 04/20/2026).

[7] R. Blaser and P. Fryzlewicz. *Random Rotation Ensembles*. London School of Economics, 2016. URL: https://dl.acm.org/doi/pdf/10.5555/2946645.2946649 (visited on 04/21/2026).